

1. Perform primitive operations – Union & Intersection on Fuzzy sets.**Code:**

```
#Fuzzy sets operation
A,B,Y,Z={"a": 0.2, "b": 0.3, "c": 0.6, "d": 0.6},{ "a": 0.9, "b":
0.9, "c": 0.4, "d": 0.5},dict(),dict()
print('The First Fuzzy Set is :', A)
print('The Second Fuzzy Set is :', B)
for A_key, B_key in zip(A, B):
    A_value = A[A_key]
    B_value = B[B_key]
    if A_value > B_value:
        Y[A_key] = A_value
        Z[B_key] = B_value
    else:
        Y[B_key] = B_value
        Z[A_key] = A_value

print('Fuzzy Set Union is : ', Y)
print('Fuzzy Set Intersection is: ',Z)
```

Output:

```
The First Fuzzy Set is : {'a': 0.2, 'b': 0.3, 'c': 0.6, 'd': 0.6}
The Second Fuzzy Set is : {'a': 0.9, 'b': 0.9, 'c': 0.4, 'd':
0.5}
Fuzzy Set Union is :  {'a': 0.9, 'b': 0.9, 'c': 0.6, 'd': 0.6}
Fuzzy Set Intersection is:  {'a': 0.2, 'b': 0.3, 'c': 0.4, 'd':
0.5}
```

Numerical:

(i)

$$A = \left\{ \frac{0.2}{a} + \frac{0.3}{b} + \frac{0.6}{c} + \frac{0.6}{d} \right\}$$

$$B = \left\{ \frac{0.9}{a} + \frac{0.9}{b} + \frac{0.4}{c} + \frac{0.5}{d} \right\}$$

$$A \cup B = \max (\mu_A, \mu_B)$$

$$A \cap B = \min (\mu_A, \mu_B)$$

$$\Rightarrow A \cup B = \left\{ \frac{0.9}{a} + \frac{0.9}{b} + \frac{0.6}{c} + \frac{0.6}{d} \right\}$$

$$\Rightarrow A \cap B = \left\{ \frac{0.2}{a} + \frac{0.3}{b} + \frac{0.4}{c} + \frac{0.5}{d} \right\}$$

2. Perform primitive operations – Complement, Difference and Symmetric Difference on Fuzzy sets.

Code:

```
import fuzzysets as fs
q,w=dict(),dict()
n=int(input("Enter how many elements:"))
for i in range(0,n):
    a=input("Enter elements of set:")
    q[a]=float(input("Enter membership function of set A:"))
    w[a]=float(input("Enter membership function of set B:"))

A,B=fs.FiniteFuzzySet(q),fs.FiniteFuzzySet(w)

def diff(X,Y):
    return X.t_norm(fs.complement(Y))

print("Fuzzy sets:\n"+"A="+str(A)+"\nB="+str(B))
print("Intersection:\n"+str(A.t_norm(B)))
print("Union:\n"+str(A.s_norm(B)))
print("A Complement:\n"+str(fs.complement(A))+"\nB Complement:\n"+str(fs.complement(B)))
print("A|B=\n"+str(diff(A,B)))
print("A Symm. Diff. B=\n"+str(diff(A,B).s_norm(diff(B,A))))
```

Output:

```
Enter how many elements:3
Enter elements of set:q
Enter membership function of set A:.1
Enter membership function of set B:.7
Enter elements of set:q
Enter membership function of set A:.4
Enter membership function of set B:.6
Enter elements of set:r
Enter membership function of set A:.2
Enter membership function of set B:.9
Fuzzy sets:
A=r/0.20 + q/0.40
B=r/0.90 + q/0.60
Intersection:
r/0.20 + q/0.40
Union:
r/0.90 + q/0.60
A Complement:
r/0.80 + q/0.60
B Complement:
r/0.10 + q/0.40
A|B=
```

$$\begin{aligned}
 &r/0.10 + q/0.40 \\
 &\text{A Symm. Diff. B=} \\
 &r/0.80 + q/0.60
 \end{aligned}$$

Numerical:

(1-b)

$$\text{Fuzzy set 1 : } A = \left\{ \frac{x}{0.2}, \frac{y}{0.3}, \frac{z}{0.4} \right\}$$

$$\text{Fuzzy set 2 : } B = \left\{ \frac{x}{0.3}, \frac{y}{0.4}, \frac{z}{0.5} \right\}$$

$$A' = \left\{ \frac{x}{0.8}, \frac{y}{0.7}, \frac{z}{0.6} \right\}$$

$$B' = \left\{ \frac{x}{0.7}, \frac{y}{0.6}, \frac{z}{0.5} \right\}$$

$$A/B = \left\{ \frac{x}{0.2}, \frac{y}{0.3}, \frac{z}{0.4} \right\}$$

$$A \Delta B = \left\{ \frac{x}{0.3}, \frac{y}{0.4}, \frac{z}{0.5} \right\}$$

3. Perform Cartesian product on Fuzzy sets.

Code:

```
n,p=int(input("Number of elements of A:")),int(input("Number of elements of B:"))
A,B,C=[],[],[]

for i in range(n):
    A.append(float(input("Element %d of A:" % (i))))
for i in range(p):
    B.append(float(input("Element %d of B:" % (i))))

print(A)
print(B)

for i in A:
    for j in B:
        C.append(min(i,j))

arr=np.array(C).reshape(len(A),len(B))
print("AxB:\n"+str(arr))
print("BxA:\n"+str(arr.T))
```

Output:

```
Number of elements of A:2
Number of elements of B:2
Element 0 of A:.3
Element 1 of A:.1
Element 0 of B:.2
Element 1 of B:.7
[0.3, 0.1]
[0.2, 0.7]
AxB:
[[0.2 0.3]
 [0.1 0.1]]
BxA:
[[0.2 0.1]
 [0.3 0.1]]
```

Numerical:

$$(3) \quad A = \left\{ \frac{0.3}{a} + \frac{0.9}{b} \right\}$$

$$B = \left\{ \frac{0.5}{1} + \frac{0.7}{2} + \frac{0.4}{3} \right\}$$

$$A \times B = \begin{matrix} a & \begin{bmatrix} 0.3 & 0.3 & 0.3 \end{bmatrix} \\ b & \begin{bmatrix} 0.5 & 0.7 & 0.1 \end{bmatrix} \end{matrix}$$

$$B \times A = \begin{matrix} 1 & \begin{bmatrix} 0.3 & 0.5 \end{bmatrix} \\ 2 & \begin{bmatrix} 0.3 & 0.7 \end{bmatrix} \\ 3 & \begin{bmatrix} 0.3 & 0.4 \end{bmatrix} \end{matrix}$$

4. Perform max-min and max-product operations on Fuzzy relations.**Code:**

```
import numpy as np

def maxMin(x, y):
    z = []
    for x1 in x:
        for y1 in y:
            z.append(max(np.minimum(x1, y1)))
    return np.array(z).reshape((x.shape[0], y.shape[1]))

def maxProduct(x, y):
    z = []
    for x1 in x:
        for y1 in y:
            z.append(max(np.multiply(x1, y1)))
    return np.array(z).reshape((x.shape[0], y.shape[1]))

m,n,p,q=int(input("Relation 1:\nNumber of rows: ")),int(input("Number of columns: ")),int(input("Relation 2:\nNumber of rows: ")),int(input("Number of columns: "))
if(n!=p):
    print("Invalid order")
else:
    a,b=[],[]

    for i in range(m):
        aa=[]
        for j in range(n):
            aa.append(float(input()))
        a.append(aa)
    for i in range(p):
        bb=[]
        for j in range(q):
            bb.append(float(input()))
        b.append(bb)

R1,R2=np.array(a).reshape(m,n),np.array(b).reshape(p,q)

print("R1:")
print(R1)
print("R2:")
```



```
print(R2)
print("R1oR2 => Max-Min :\n" + str(maxMin(R1,R2)) + "\n")
print("R1oR2 => Max-Product :\n" + str(maxProduct(R1,)) + "\n\n")
```

Output:

```
Relation 1:
Number of rows: 2
Number of columns: 2
Relation 2:
Number of rows: 2
Number of columns: 2
.1
.2
.3
.4
.5
.6
.7
.8
R1:
[[0.1 0.2]
 [0.3 0.4]]
R2:
[[0.5 0.6]
 [0.7 0.8]]
R1oR2 => Max-Min :
[[0.2 0.2]
 [0.4 0.4]]

R1oR2 => Max-Product :
[[0.12 0.16]
 [0.24 0.32]]
```

Numerical:

1) max-min composition:-

$$\begin{aligned}\mu_T(x, z_1) &= \max \{ \min [\mu_R(x, y_1), \mu_S(y_1, z_1)] \\ &\quad \min [\mu_R(x, y_2), \mu_S(y_2, z_1)] \} \\ &= \max \{ 0.6, 0.3 \} = 0.6\end{aligned}$$

$$\mu_T(x_1, z_2) = \max \{ 0.5, 0.3 \} = 0.5$$

$$\mu_T(x_1, z_3) = \max \{ 0.3, 0.3 \} = 0.3$$

$$\mu_T(x_2, z_1) = \max \{ 0.2, 0.8 \} = 0.8$$

$$\mu_T(x_3, z_2) = \max \{ 0.2, 0.4 \} = 0.4$$

$$\mu_T(x_3, z_3) = \max \{ 0.2, 0.7 \} = 0.7$$

$$T = R \circ S = \begin{matrix} & \begin{matrix} z_1 & z_2 & z_3 \end{matrix} \\ \begin{matrix} x_1 \\ x_2 \end{matrix} & \begin{bmatrix} 0.6 & 0.5 & 0.3 \\ 0.8 & 0.4 & 0.7 \end{bmatrix} \end{matrix}$$

max product $Z = R \cdot S$

$$\begin{aligned}\mu_Z(x_1, z_1) &= \max \{ (\mu_R(x_1, y_1) \cdot \mu_S(y_1, z_1)), \mu_R(x_1, y_2) \cdot \mu_S(y_2, z_1) \} \\ &= \max \{ 0.6, 0.24 \} \Rightarrow 0.6\end{aligned}$$

$$\mu_Z(x_1, z_2) = \max \{ 0.6 \times 0.5, 0.3 \times 0.4 \} \Rightarrow 0.3$$

$$\mu_Z(x_1, z_3) = \max \{ 0.6 \times 0.3, 0.3 \times 0.7 \} \Rightarrow 0.21$$

$$\mu_Z(x_2, z_1) = \max \{ 0.2 \times 1, 0.9 \times 0.8 \} \Rightarrow 0.72$$

$$\mu_Z(x_2, z_2) = \max \{ 0.2 \times 0.5, 0.7 \times 0.4 \} \Rightarrow 0.36$$

$$\mu_Z(x_2, z_3) = \max \{ 0.2 \times 0.3, 0.9 \times 0.7 \} \Rightarrow 0.63$$

$$\therefore Z = \begin{matrix} & \begin{matrix} z_1 & z_2 & z_3 \end{matrix} \\ \begin{matrix} x_1 \\ x_2 \end{matrix} & \begin{bmatrix} 0.6 & 0.3 & 0.21 \\ 0.72 & 0.36 & 0.63 \end{bmatrix} \end{matrix}$$

5. Find crisp set after performing Union, Intersection, Difference and Complement Fuzzy sets.

Code:

```
import fuzzysets as fs

q,w=dict(),dict()
n=int(input("Enter how many elements:"))
for i in range(0,n):
    a=input("Enter elements of set:")
    q[a]=float(input("Enter membership function of set A:"))
    w[a]=float(input("Enter membership function of set B:"))

A,B=fs.FiniteFuzzySet(q),fs.FiniteFuzzySet(w)

def diff(X,Y):
    return X.t_norm(fs.complement(Y))

I,U,Ac,Bc,AdiffB,AsymB=A.t_norm(B),A.s_norm(B),fs.complement(A),f
s.complement(B),diff(A,B),diff(A,B).s_norm(diff(B,A))
I=A.t_norm(B)
U=A.s_norm(B)
Ac=fs.complement(A)
Bc=fs.complement(B)
AdiffB=diff(A,B)

print("Fuzzy sets:\n"+"A="+str(A)+"\nB="+str(B))
print("Intersection:\n"+str(I))
print("Union:\n"+str(U))
print("A Complement:\n"+str(Ac)+"\nB Complement:\n"+str(Bc))
print("A|B:\n"+str(AdiffB))
print("A Symm. Diff. B:\n"+str(AsymB))

lam=float(input("Lambda: "))
print("Defuzzified: ")
print("Intersection: "+str(I.alpha_cut(alpha=lam)))
print("Union: "+str(U.alpha_cut(alpha=lam)))
print("A Complement: "+str(Ac.alpha_cut(alpha=lam)))
print("B Complement: "+str(Bc.alpha_cut(alpha=lam)))
print("A|B: "+str(AdiffB.alpha_cut(alpha=lam)))
print("A Symm. Diff. B: "+str(AsymB.alpha_cut(alpha=lam)))
```

Output:

```
Enter how many elements:3
Enter elements of set:a
Enter membership function of set A:.1
Enter membership function of set B:.3
Enter elements of set:b
Enter membership function of set A:.5
Enter membership function of set B:.3
Enter elements of set:c
Enter membership function of set A:.4
Enter membership function of set B:.8
```

```
Fuzzy sets:
A=c/0.40 + b/0.50 + a/0.10
B=c/0.80 + b/0.30 + a/0.30
Intersection:
c/0.40 + b/0.30 + a/0.10
Union:
c/0.80 + b/0.50 + a/0.30
A Complement:
c/0.60 + b/0.50 + a/0.90
B Complement:
c/0.20 + b/0.70 + a/0.70
A|B:
c/0.20 + b/0.50 + a/0.10
A Symm. Diff. B:
c/0.60 + b/0.50 + a/0.30
Lambda: .3
Defuzzified:
Intersection: {'c', 'b'}
Union: {'c', 'b', 'a'}
A Complement: {'c', 'b', 'a'}
B Complement: {'b', 'a'}
A|B: {'b'}
A Symm. Diff. B: {'c', 'b', 'a'}
```

Numerical:

$$(5) \quad A = \left\{ \frac{0}{a} + \frac{0.5}{b} + \frac{0.65}{c} + \frac{0.85}{d} \right\}$$

$$B = \left\{ \frac{0}{a} + \frac{0.45}{b} + \frac{0.6}{c} + \frac{0.8}{d} \right\}$$

$$\text{Union} \Rightarrow \left\{ \frac{0}{a} + \frac{0.5}{b} + \frac{0.65}{c} + \frac{0.85}{d} \right\}$$

$$\text{Intersection} \Rightarrow \left\{ \frac{0}{a} + \frac{0.45}{b} + \frac{0.6}{c} + \frac{0.8}{d} \right\}$$

$$\neq A - B = A \cap \bar{B}$$

$$\bar{B} = 1 - B \Rightarrow \left\{ \frac{1}{a} + \frac{0.55}{b} + \frac{0.4}{c} + \frac{0.2}{d} \right\}$$

$$A \cap \bar{B} \Rightarrow \left\{ \frac{0}{a} + \frac{0.5}{b} + \frac{0.4}{c} + \frac{0.2}{d} \right\}$$

$$\neq B - A = B \cap \bar{A}$$

$$\bar{A} = 1 - A = \left\{ \frac{1}{a} + \frac{0.5}{b} + \frac{0.35}{c} + \frac{0.15}{d} \right\}$$

$$B \cap \bar{A} = \left\{ \frac{0}{a} + \frac{0.45}{b} + \frac{0.35}{c} + \frac{0.15}{d} \right\}$$

$$A \Delta B = \left\{ \frac{0}{a} + \frac{0.5}{b} + \frac{0.4}{c} + \frac{0.2}{d} \right\}$$

Crispsat:-

$$\lambda = 0.5$$

$$A_{0.5} = \{ b, c, d \}$$

$$B_{0.5} = \{ c, d \}$$

6. Perform Lambda cut on Union and Intersection for given Fuzzy relations.**Code:**

```
import numpy as np

def inp(n,p):
    a=[]
    for i in range(n):
        l1=[]
        for j in range(p):
            l1.append(float(input()))
        a.append(l1)
    return a

def foo(l,z,n,p):
    a=[]
    for i in range(n):
        uu=[]
        for j in range(p):
            if(U[i][j]<z):
                uu.append(0)
            else:
                uu.append(1)
        a.append(uu)
    return a

def fool(a,b,q,n,p):
    if(q=="Union"):
        U=[]
        for i in range(n):
            uu=[]
            for j in range(p):
                uu.append(max(a[i][j],b[i][j]))
            U.append(uu)
        return U
    if(q=="Intersection"):
        I=[]
        for i in range(n):
            uu=[]
            for j in range(p):
                uu.append(min(a[i][j],b[i][j]))
            I.append(uu)
        return I

n,p =int(input("No of rows: ")),int(input("No of columns: "))
print("Input Relation 1")
```

```
a=inp(n,p)
print("Input Relation 2")
b=inp(n,p)

print("Relation 1")
print(np.array(a).reshape(n,p))
print("Relation 2")
print(np.array(b).reshape(n,p))
U=fool(a,b,"Union",n,p)
print("Union: ")
print(np.array(U).reshape(n,p))

I=fool(a,b,"Intersection",n,p)
print("Intersection: ")
print(np.array(I).reshape(n,p))

z=float(input("Lambda: "))

U1,I1=foo(U,z,n,p),foo(I,z,n,p)
print("After defuzzification:\nUnion: ")
print(np.array(U1).reshape(n,p))
print("Intersection: ")
print(np.array(I1).reshape(n,p))
```

Output:

```
No of rows: 2
No of columns: 2
Input Relation 1
.6
.3
.7
.2
Input Relation 2
.9
0
1
.1
Relation 1
[[0.6 0.3]
 [0.7 0.2]]
Relation 2
[[0.9 0. ]
 [1.  0.1]]
Union:
[[0.9 0.3]
 [1.  0.2]]
Intersection:
```

```
[[0.6 0. ]  
 [0.7 0.1]]  
Lambda: .3  
After defuzzification:  
Union:  
[[1 1]  
 [1 0]]  
Intersection:  
[[1 1]  
 [1 0]]
```


Numerical:

$$(c) \quad S = \begin{bmatrix} 0.9 & 0.2 & 0 & 0.5 \\ 0.8 & 0.4 & 0.4 & 0.6 \\ 0.9 & 0.6 & 0.5 & 0.8 \\ 0.8 & 0.7 & 0.6 & 0.4 \end{bmatrix}$$

$$T = \begin{bmatrix} 0.4 & 0.1 & 0.6 & 0.1 \\ 0.8 & 1 & 0.5 & 0.7 \\ 1 & 0.3 & 0 & 0.9 \\ 0.8 & 0.9 & 0.9 & 0.7 \end{bmatrix}$$

$$S \cup T \Rightarrow \begin{bmatrix} 0.9 & 0.2 & 0.6 & 0.8 \\ 0.8 & 1 & 0.5 & 0.7 \\ 0.9 & 0.6 & 0.5 & 0.9 \\ 0.8 & 0.9 & 0.9 & 0.7 \end{bmatrix}$$

$$\lambda = 0.6$$

$$(S \cup T)_{0.6} \Rightarrow \begin{bmatrix} 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

$$S \cap T \Rightarrow \begin{bmatrix} 0.4 & 0.2 & 0 & 0.5 \\ 0.8 & 0.4 & 0.4 & 0.6 \\ 0.9 & 0.3 & 0.3 & 0.8 \\ 0.8 & 0.7 & 0.6 & 0.4 \end{bmatrix}$$

$$(S \cap T)_{0.6} \Rightarrow \begin{bmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

7. Program to determine whether the Fuzzy relations are tolerance relation or not.**Code:**

```
l,n=[],int(input("Enter order of the fuzzy relation (Square Matrix): "))
print("Enter "+str(n*n)+" elements: ")
for i in range(n):
    l1=[]
    for j in range(n):
        l1.append(float(input()))
    l.append(l1)
r=np.array(l)
print("Given relation: \n"+str(r))
print("Tolerance Relation? ")
if(np.array_equal(r,r.T)):
    print("Yes")
else:
    print("No")
```

Output:

```
Enter order of the fuzzy relation (Square Matrix): 2
Enter 4 elements:
.1
.2
.3
.4
Given relation:
[[0.1 0.2]
 [0.3 0.4]]
Tolerance Relation?
```

Prog 10 : Find Net weight of a multi-input neuron & determine whether neuron is fired or not for the given threshold value and Activation Function.

Code: (for a particular threshold and activation functions)

```
th=float(input("Enter threshold: "))
# Identity
print("Identity function: f(x)=x: ")
print(yin)

print("Binary Step function: \n")
if(yin<th):
    print(0)
else:
    print(1)

# print(0 if yin<th else 1)

print("Bipolar Step function: ")
if(yin<th):
    print(-1)
else:
    print(1)

print("Ramp: ")
if(yin>1):
    print(1)
elif(yin<0):
    print(0)
else:
    print(yin)

# Sigmoidal

# Assuming lambda = 1

print("Binary Sigmoidal: ")

ys=1/(1+pow(e,-yin))
print(ys)

print("Bipolar Sigmoidal: ")
print((2*ys)-1)
```

Code: (for Net weight and Multi input neuron)

```
from math import pow, e
n,x,w=int(input("Enter number of input lines: ")),[],[]

# x.append(1)
# w.append(0)

# print("Enter %d weights: " %(n))
for i in range(n+1):
    x.append(float(input("Enter x" +str(i)+" :\n")))
    w.append(float(input("Enter w" +str(i)+" :\n")))

# print(x,w)

yin=0

for i in range(n+1):
    yin+=x[i]*w[i]

# print(w,x,yin)
print("Inputs: "+str(x)+"\nWeights: "+str(w)+"\nNet input: "+str(yin))
```

Output:

```
Enter number of input lines: 2
Enter x0:
1
Enter w0:
0
Enter x1:
1
Enter w1:
.2
Enter x2:
1
Enter w2:
.2
Inputs: [1.0, 1.0, 1.0]
Weights: [0.0, 0.2, 0.2]
Net input: 0.4
```


Prog 10 : Find Net weight of a multi-input neuron & determine whether neuron is fired or not for the given threshold value and Activation Function.

Code: (for Net weight and Multi input neuron)

```
from math import pow, e
n,x,w=int(input("Enter number of input lines: ")),[],[]

# x.append(1)
# w.append(0)

# print("Enter %d weights: " %(n))
for i in range(n+1):
    x.append(float(input("Enter x" +str(i)+" ":"\n"))))
    w.append(float(input("Enter w" +str(i)+" ":"\n"))))

# print(x,w)

yin=0

for i in range(n+1):
    yin+=x[i]*w[i]

# print(w,x,yin)
print("Inputs: "+str(x)+"\nWeights: "+str(w)+"\nNet input: "+str(yin))
```

Output:

```
Enter number of input lines: 2
Enter x0:
1
Enter w0:
0
Enter x1:
1
Enter w1:
.2
Enter x2:
1
Enter w2:
.2
Inputs: [1.0, 1.0, 1.0]
Weights: [0.0, 0.2, 0.2]
Net input: 0.4
```


Code: (for a particular threshold and activation functions)

```
th=float(input("Enter threshold: "))
# Identity
print("Identity function: f(x)=x: ")
print(yin)

print("Binary Step function: \n")
if(yin<th):
    print(0)
else:
    print(1)

# print(0 if yin<th else 1)

print("Bipolar Step function: ")
if(yin<th):
    print(-1)
else:
    print(1)

print("Ramp: ")
if(yin>1):
    print(1)
elif(yin<0):
    print(0)
else:
    print(yin)

# Sigmoidal

# Assuming lambda = 1

print("Binary Sigmoidal: ")

ys=1/(1+pow(e,-yin))
print(ys)

print("Bipolar Sigmoidal: ")
print((2*ys)-1)
```

Output:

```
Enter threshold: .5
Identity function: f(x)=x:
0.4
Binary Step function:

0
Bipolar Step function:
-1
Ramp:
0.4
```

Binary Sigmoidal:
0.598687660112452
Bipolar Sigmoidal:
0.197375320224904

Numerical:

